MCP服务[1]

1 介绍[2]

2 技术选型

2.1 antvis/mcp-server-chart

2.2 apconw/gpt-vis-api

2.3 MCPHub

2.4 如何编写自己的MCP服务

3 gpt-vis-api部署

3.1 本地测试

3.2 Dify测试

3.3 Qwen3:32b

3.4 小模型调用MCP

3.4.1 问题1

3.4.2 问题2

3.4.3 总结

4 调用细节

5 替代方案

插件安装过程中遇到异常应该如何处理?

1 介绍[3]

`MCP` 全称 **Model Context Protocol** （中文名：模型上下文协议），是由 Anthropic 于 2024 年 11 月推出的开源协议。**旨在解决不同大语言模型（LLM）与不同外部工具集成的标准化问题** 。[4]

MCP 核心是让大模型能方便地调用多个工具，如本地文件、数据库、各网站数据等。

dify v1.6.0 支持双向MCP[5]

1. 可以直接从 Dify 调用任何 MCP 服务。无需局限于现有的 Dify 插件，而是可以利用强大的 MCP 服务器。[6]
2. 支持将你的 Dify工作流作为 MCP 服务，无缝集成其他客户端。使得这些工具能够像原生扩展一样直接与您的 Dify 应用交互。[7]



Greetings, Dify builders! 3

We've packed the early-summer stretch (Jun 1 – Jul 10) with upgrades you can ship today. 4

双向MCP(v1.6.0) 5

TL;DR: 一键调用任何MCP服务器或发布您自己的服务器 6

- **Two-way MCP (v1.6.0):** Call any MCP server or publish your own in one click. 7
- **Enhanced Workflow Debugging (v1.5.0):** Inspect variables, rerun single nodes, fix fast. 8
- **Azure AI Content Safety Plugin:** On-prem moderation for text + images. 9
- **100 K GitHub Stars:** Meet the 17 contributors powering the journey. 10

Built-in Two-Way MCP (v1.6.0) 11

Agents and Workflows now speak MCP natively. Add Linear, Notion, Zapier (8 000+ apps) as tools, or expose your own Dify apps as MCP servers for any client to invoke—no glue code required. 12

调用的是魔塔的MCP广场中的服务 13

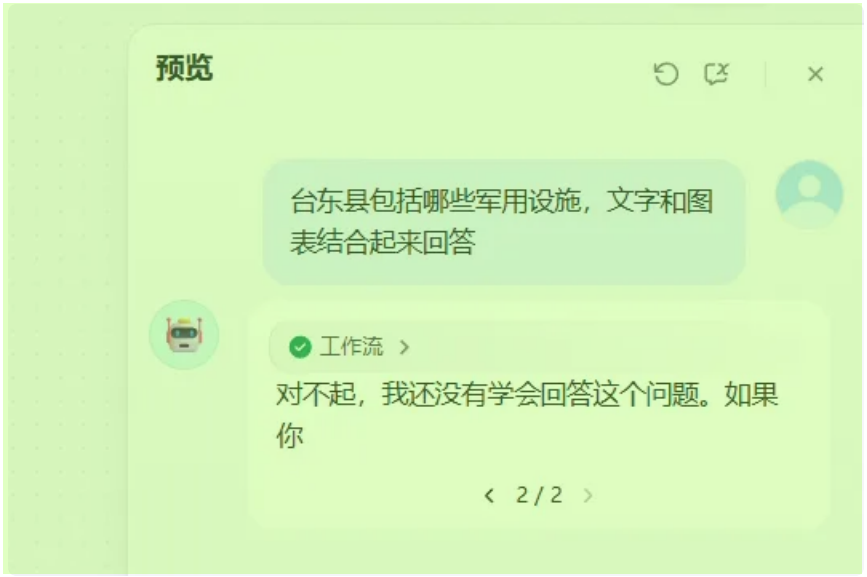


1

Dify快速接入本地MCP服务示例 2

知 超实用！Dify快速接入本地MCP服务 3

目前遇到的问题，本地deepseek 70b 无法成功调用MCP服务 4



5

2 技术选型 6

2.1 antvis/mcp-server-chart [2]

之前根据【MCP-server-chart部署及踩坑建议】-CSDN博客部署了ant提供的mcp服务mcp-server-chart [3]

GitHub - antvis/mcp-server-chart: 🤖 A visualization mcp contains 25+ visual charts using @antvis. Using for chart generation and data analysis. [4]

发现项目issues中作者提到这个需要一个远程服务 [5]



[6] [7] [8] [9] [10]

如果需要私有化还需部署 [11]

GitHub - antvis/GPT-Vis: 🤖 GPT Vision, Open Source Vision components for GPTs, generative AI, and LLM projects. Not only UI Components. [12]



[13] [14] [15] [16] [17] [18]



[19] [20] [21] [22] [23] [24] [25]

具体配置方法 [26]

Plain Text [1]

```
{
  "mcpServers": {
    "mcp-server-chart": {
      "command": "npx",
      "args": ["-y", "@antv/mcp-server-chart"],
      "env": {
        "VIS_REQUEST_SERVER": "<您的私有服务地址>"
      }
    }
  }
}
```

技术实现步骤 [2]

1. **部署私有服务**：使用 AntV 的 `GPT-Vis-SSR` 项目在自己服务器部署 HTTP 服务 [3]
2. **配置环境变量**：将私有服务的 URL 设置为 `VIS_REQUEST_SERVER` [4]
3. **API 接口规范**：[5]
   - **方法**: POST [6]
   - **参数**: 图表配置数据，如 `{"type":"line", "data":[...]}` [7]
   - **返回**: 包含生成图表图片URL的对象 [8]

[9]「一语成图」：GPT-Vis × MCP 让大模型「说句话就出可视化」-CSDN博客 [10]

2.2 apconw/gpt-vis-api [11]

如何使用 AntV 的 GPT-Vis-SSR项目在自己服务器部署 HTTP 服务? [12]

apconw提供了GPT-VIS-API及示例应用项目 [13]

GitHub - apconw/gpt-vis-api: GPT-VIS-API 是一个轻量级图表生成服务，旨在解决 [antv/mcp-server-chart](https://github.com/antvis/mcp-server-chart) 在私有化部署方面的局限性。该服务接收数据请求，生成图表图像，上传到 MinIO 对象存储，并返回带有效期的预签名访问链接。 [14]

GPT-VIS-API [1]

> 基于 Node.js 和 Express 构建的图表生成服务，支持将生成的图表上传至 MinIO 并返回预签名 URL。 [2]

📌 项目概述 [3]

GPT-VIS API 是一个轻量级图表生成服务，旨在解决 antv/mcp-server-chart 在私有化部署方面的局限性。该服务接收数据请求，生成图表图像，上传到 MinIO 对象存储，并返回带有效期的预签名访问链接。 [4]

主要功能包括： [5]

- 接收 JSON 格式的数据请求 [6]
- 使用 AntV/G 生成图表（如折线图、柱状图等） [7]
- 将图表图片上传至 MinIO 存储 [8]
- 返回图片的预签名 URL 供前端访问 [9]

GitHub – apconw/sanic-web: 一个轻量级、支持全链路且易于二次开发的大模型应用项目(Large Model Data Assistant) 支持DeepSeek/Qwen3等大模型 基于 Dify 、LangChain/LangGraph、Ollama&Vllm、Sanic 和 Text2SQL 📊 等技术构建的一站式大模型应用开发项目，采用 Vue3、TypeScript 和 Vite 5 打造现代UI。它支持通过 ECharts 📈 实现基于大模型的数据图形化问答，具备处理 CSV 文件 📂 表格问答的能力。同时，能方便对接第三方开源 RAG 系统 检索系统 🌐等，以支持广泛的通用知识问答。 [10]

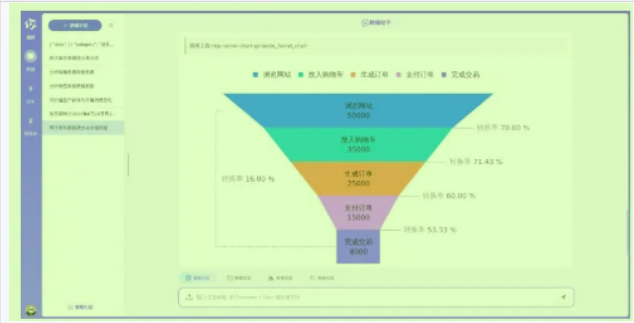
特性 1

- **多智能体**: 支持多模型、多智能体+MCP协同任务 2
- **大模型交互**: 全面支持主流大模型交互框架，如 Dify 与 LangChain 3
- **核心技术栈**：Dify + Ollama + RAG + (Qwen3/DeepSeek) + Text2SQL + MCP + Neo4J + LangGraph 4
- **UI 框架**： Vue 3 + TypeScript + Vite 5 5
- **数据问答**：集成 ECharts/AntV大模型实现Text2SQL轻量级的图形化数据问答展示 6
- **表格问答**：支持 CSV格式文件的上传与基于大模型总结预处理和Text2SQL的表格数据问答 7
- **通用问答**：支持通用数据形式问答基于对接三方RAG系统+公网检索模式 8
- **应用架构**：作为一个轻量级全链路一站式大模型应用开发框架方便扩展落地 9
- **灵活部署**：支持大模型应用开发各依赖组件docker-compose一键拉起快速部署零配置 10

案例展示 11


12


13


14


15

Environment Requirements 1

Before you start, make sure your development environment meets the following minimum requirements: 2

- **Operating System**: Windows 10/11, macOS M series, Centos/Ubuntu 3
- **GPU**: For local deployment with Ollama, an NVIDIA GPU is recommended. Alternatively, you can use the CPU mode or purchase an API key from the public network. 4
- **Memory**: 8GB+ 5

Prerequisites 6

- Python 3.8+ 7
- Poetry 1.8.3+ 8
- Dify 0.7.1+ 9
- Mysql 8.0+ 10
- Node.js 18.12.x+ 11
- Pnpm 9.x 12

Large Model Deployment 13

- Refer to Ollama Deployment 14
- Models: Qwen2.5 7B model 15
- Models: DeepSeek R1 7B model 16
- Alibaba Cloud Public Network API Key 17

apconw的部署方案 18

用 AntV MCP Server Chart 赋能大模型 —— 从零构建可视化智能体，图表生成效率飙升！大模型入门到精通，收藏这篇就足够了！_mcp-server-chart-CSDN博客 19

2.3 MCPHub 20

本地部署、一键安装、分组路由：MCPHub 重塑 MCP 服务器体验 21

为什么要使用 MCPHub 22

1. 复杂的环境依赖与配置 23

- • MCP 服务器常依赖 Node.js、Python 等多种运行时，需手动维护大量命令、参数和环境变量。 24
- • MCPHub 内置 MCP 服务器市场，包含众多常用 MCP 服务器，支持一键安装和自动配置，简化了环境搭建过程。 25
- • 通过 Docker 部署，MCPHub 可在任何支持 Docker 的平台上运行，避免了环境不一致的问题。 26



无趣

大佬您好，我把MCPHub部署好了，添加本地的MCP服务也添加成功了，但是用Dify连接MCPHub后，调用不到MCP工具

06-04 · 日本 回复 喜欢



孙青扬 作者

dify 连接不太稳定，要看具体使用哪种方式（sse/mcp），可以结合日志分析一下

06-05 · 江苏 回复 喜欢



Amoypark

非常感谢，本地部署，不能联网，是不是需要先在联网机台进行下载所需的插件，然后再次打包成镜像文件到离线服务器，这样是不是就可以使用?

04-23 · 江苏

回复 喜欢



孙青扬 作者

是的，他们都是这样干的

04-23 · 江苏

回复 喜欢

用MCPHub搭建本地自己的MCP平台 – 53AI–AI知识库|大模型知识库|大模型训练|智能体开发

2.4 如何编写自己的MCP服务

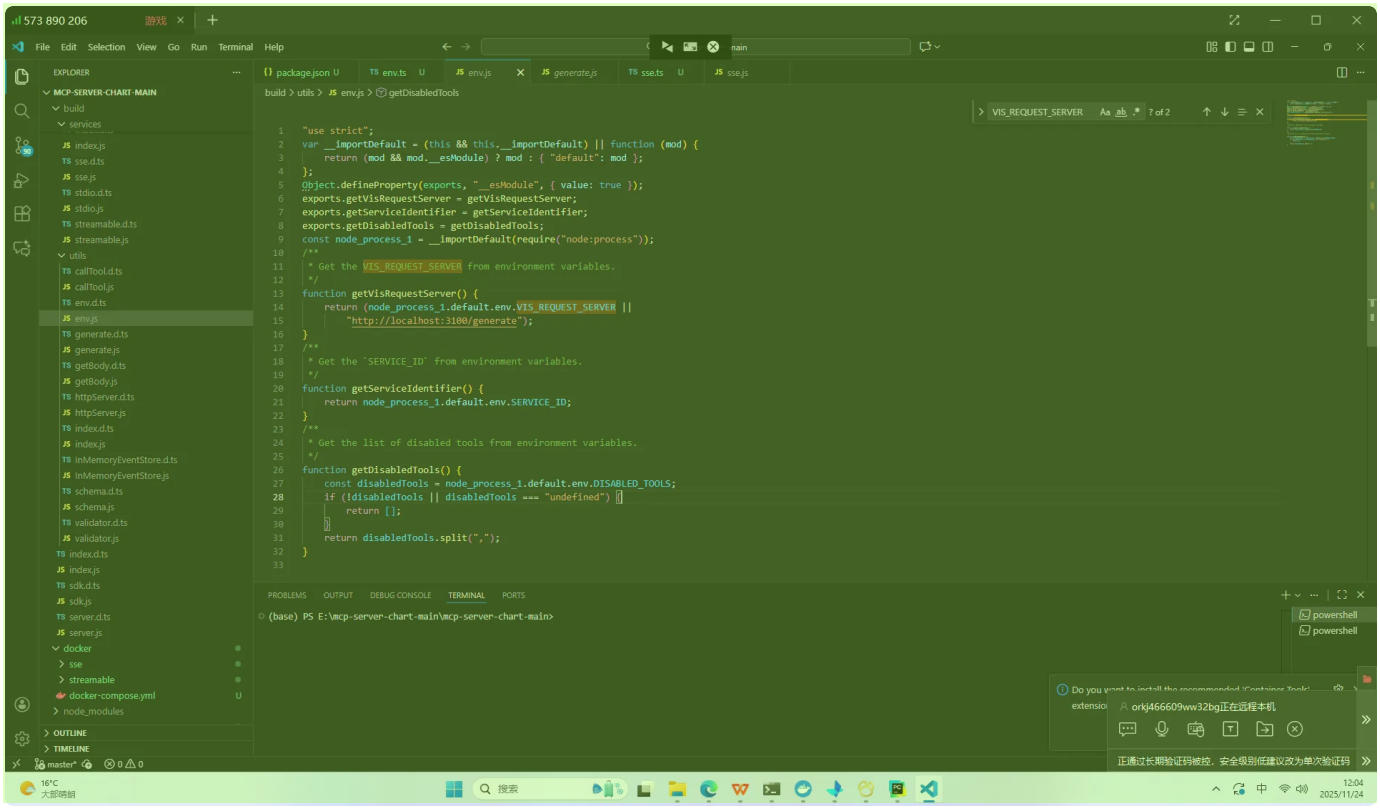
mcp–server案例分享–手搓一个MCP–server，手把手教你从本地开发到云部署全流程实战 – 文章 – 开发者社区 – 火山引擎

下一步计划在本地手搓一个MCP服务，看在dify中ollama的deepseek能否调用这个服务，如果可以，说明就是模型的算力不够，导致不能成功生成图形。

3 gpt–vis–api部署

部署了apconw/gpt–vis–api服务，可以接收 JSON 格式的数据请求，使用 AntV/G 生成图表（如折线图、柱状图等），将图表图片上传至 MinIO 存储并返回图片的预签名 URL 供前端访问

在mcp–server–chart进行apconw/gpt–vis–api服务的配置



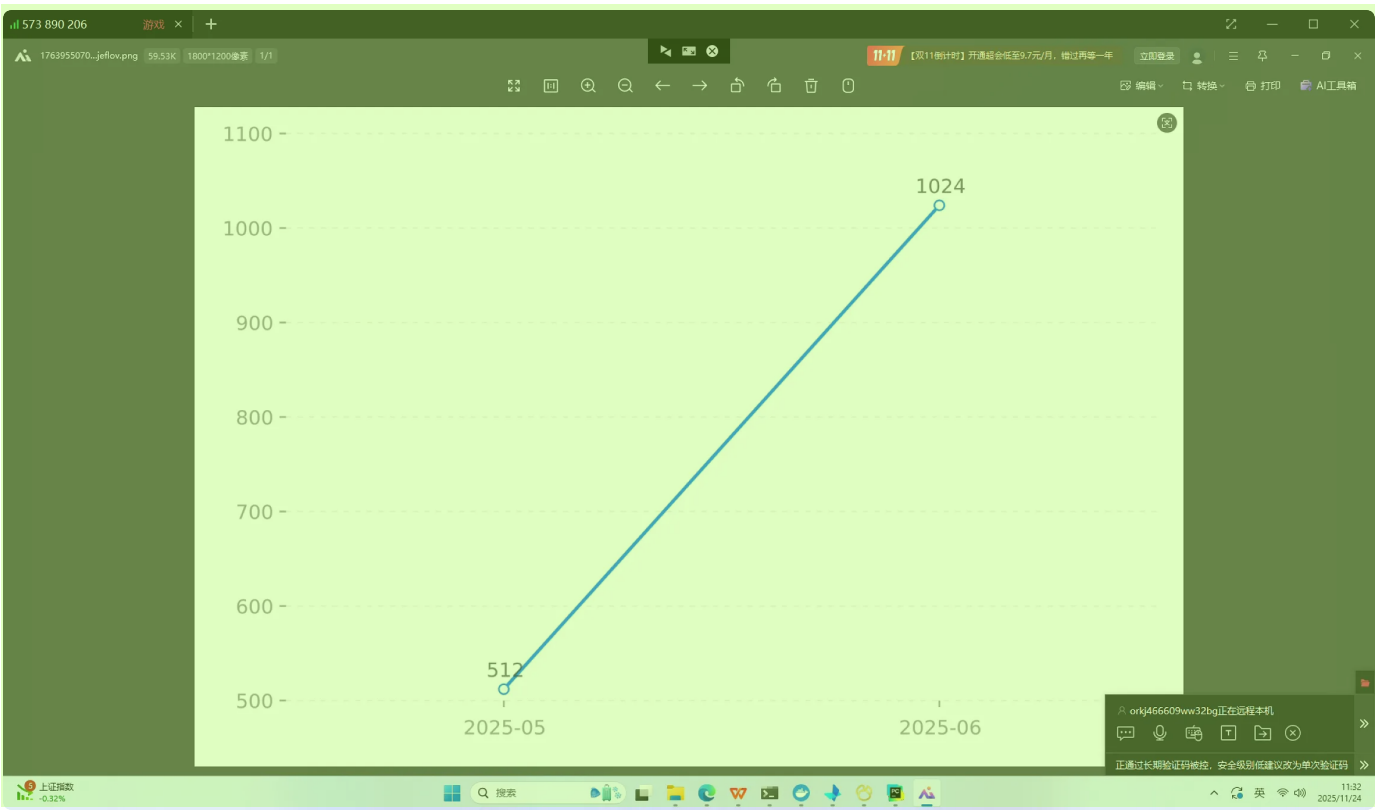
1

3.1 本地测试 2

测试用例1 3

```
E:\gpt-vis-api-main\gpt-vis-api-main>curl -X POST http://localhost:3100/generate -H "Content-Type: application/json" -d
"{\"type\": \"line\",\"data\": [{\"time\": \"2025-05\",\"value\": 512},{\"time\": \"2025-06\",\"value\": 1024}]}"
{"success":true,"resultObj":"http://localhost:19200/chart-images/charts/1763955070350_nvjeflov.png","message":"图表生成
并上传成功"}
E:\gpt-vis-api-main\gpt-vis-api-main>
```

minio控制台http://localhost:19201 admin/12345678



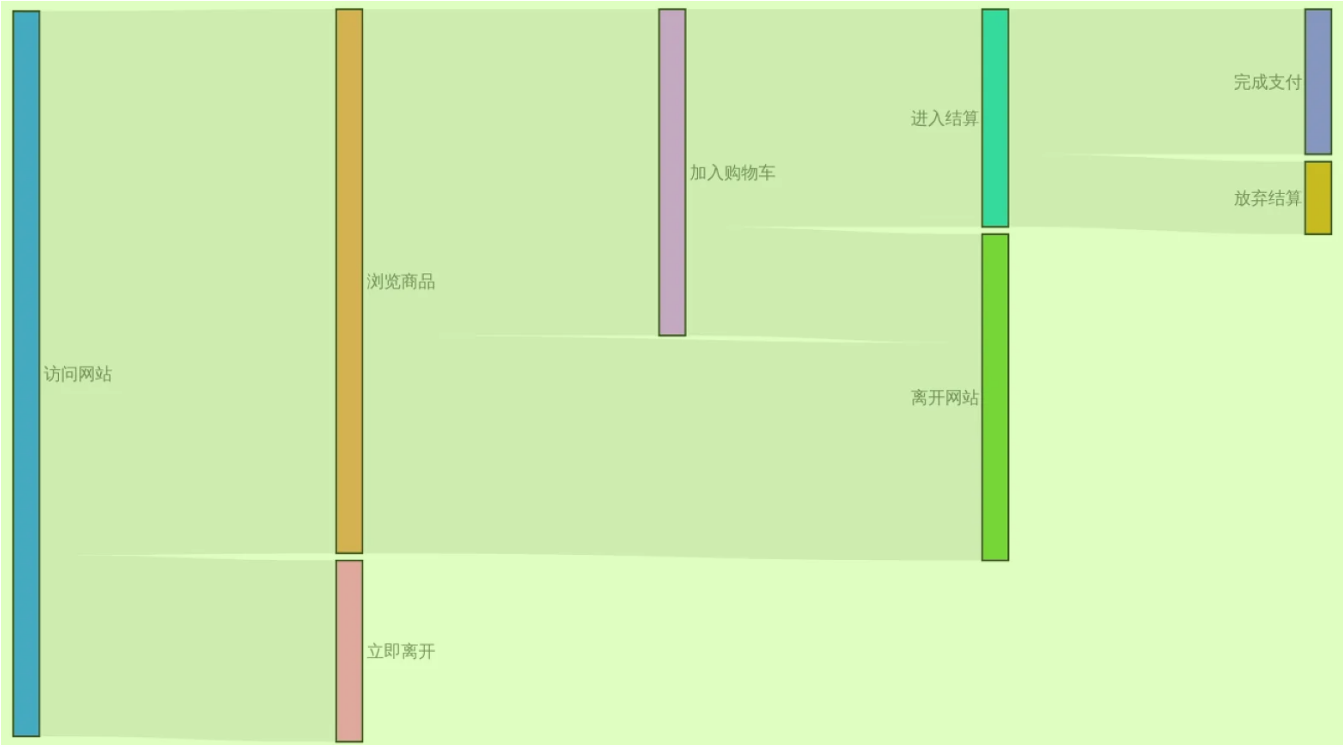
1

测试用例2 2

```
E:\gpt-vis-api-main\gpt-vis-api-main>curl -X POST http://localhost:3100/generate -H "Content-Type: application/json" -d
"{\"type\": \"line\",\"data\": [{\"time\": \"2025-05\",\"value\": 512},{\"time\": \"2025-06\",\"value\": 1024}]}"
{"success":true,"resultObj":"http://localhost:19200/chart-images/charts/1763955070350_nvjeflov.png","message":"图表生成
并上传成功"}
E:\gpt-vis-api-main\gpt-vis-api-main>curl -X POST http://localhost:3100/generate -H "Content-Type: application/json" -d
"{\"type\": \"sankey\",\"data\": [{\"source\": \"访问网站\",\"target\": \"浏览商品\",\"value\": 1500},{\"source\": \"浏
览商品\",\"target\": \"加入购物车\",\"value\": 900},{\"source\": \"加入购物车\",\"target\": \"进入结算\",\"value\": 600}
,{\"source\": \"进入结算\",\"target\": \"完成支付\",\"value\": 400},{\"source\": \"进入结算\",\"target\": \"放弃结算\",\
"value\": 200},{\"source\": \"加入购物车\",\"target\": \"离开网站\",\"value\": 300},{\"source\": \"浏览商品\",\"target\"
: \"离开网站\",\"value\": 600},{\"source\": \"访问网站\",\"target\": \"立即离开\",\"value\": 500}],\"nodeAlign\": \"cent
er\",\"style\": {\"backgroundColor\": \"#f8f9fa\",\"palette\": [\"#4e79a7\",\"#f28e2b\",\"#e15759\",\"#76b7b2\",\"#59a14
f\",\"#edc948\",\"#b07aa1\",\"#9c755f\"],\"texture\": \"default\"},\"theme\": {\"fontFamily\": \"WenQuanYi Zen Hei\"},\"
width\": 800,\"height\": 500,\"title\": \"用户购买旅程桑基图\",\"source\": \"mcp-server-chart\"}"
{"success":true,"resultObj":"http://localhost:19200/chart-images/charts/1763955231693_ltlm2muq.png","message":"图表生成
并上传成功"}
```

3

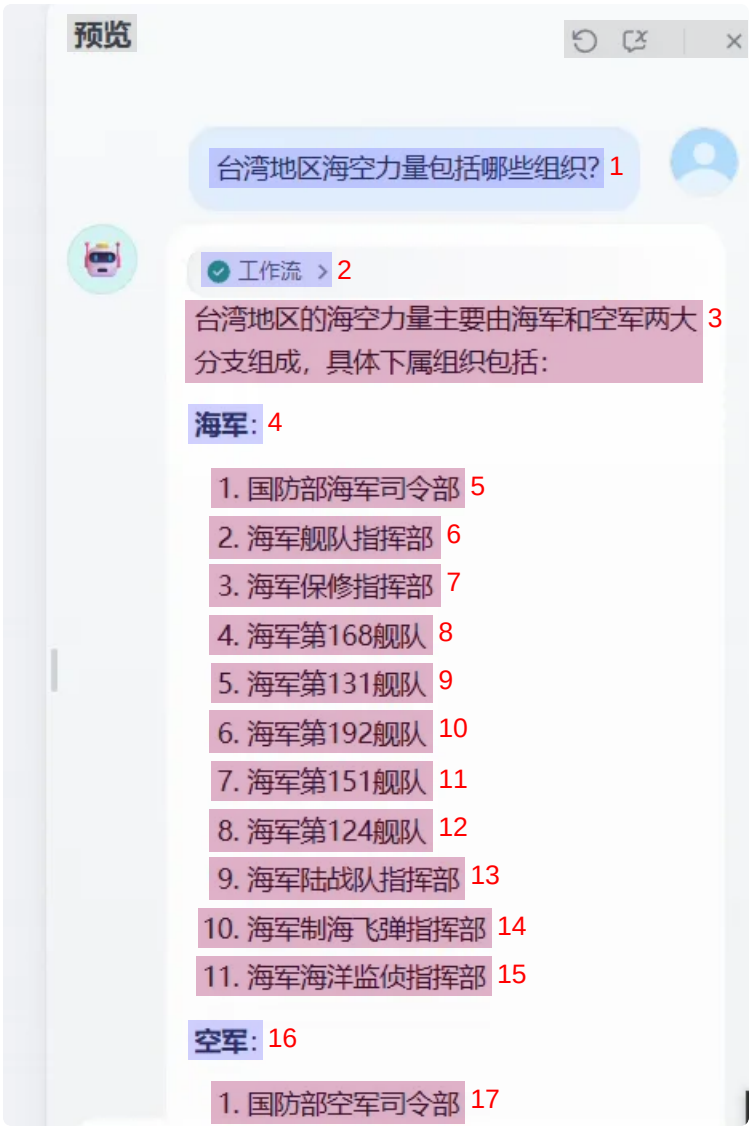
用户购买旅程桑基图 1

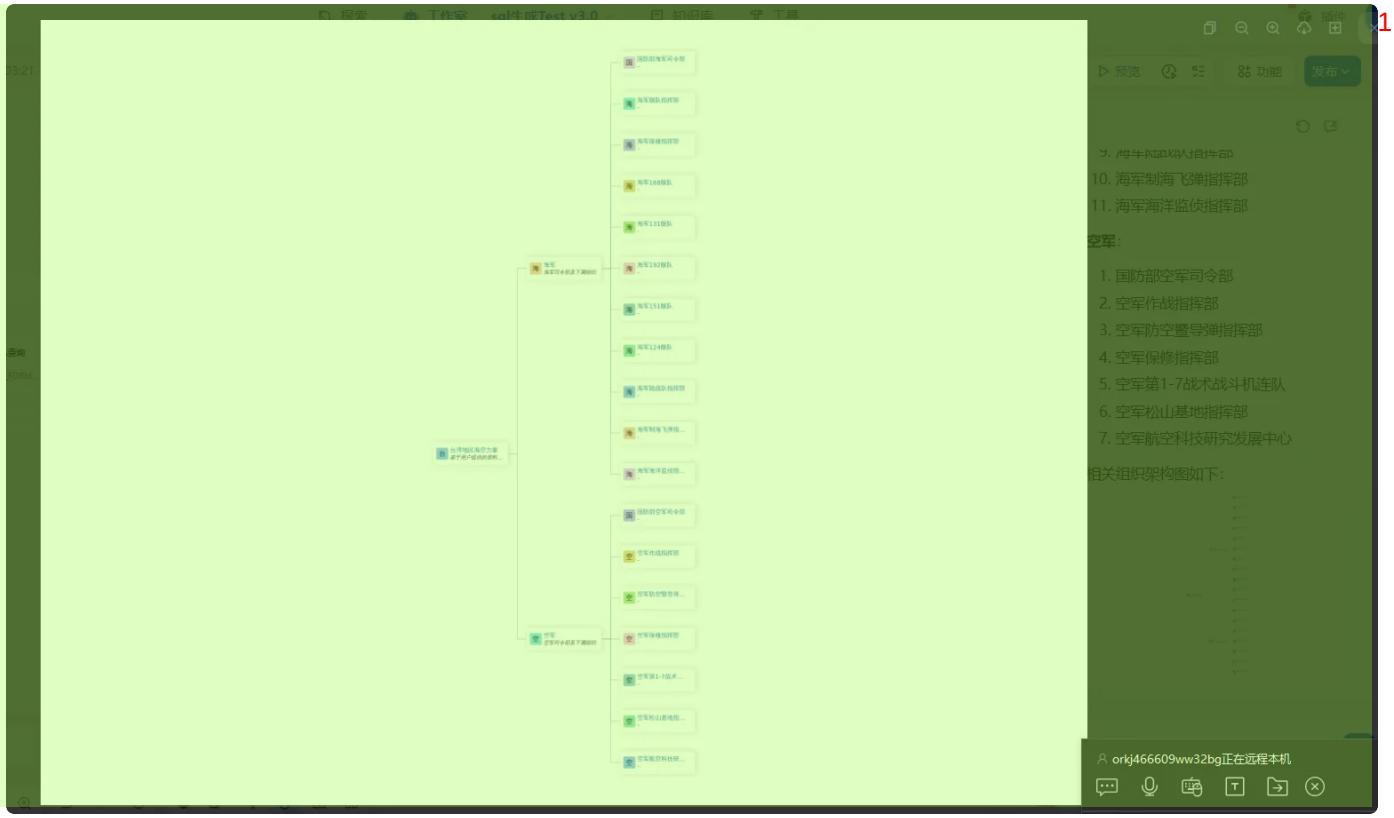


2

3.2 Dify测试 3

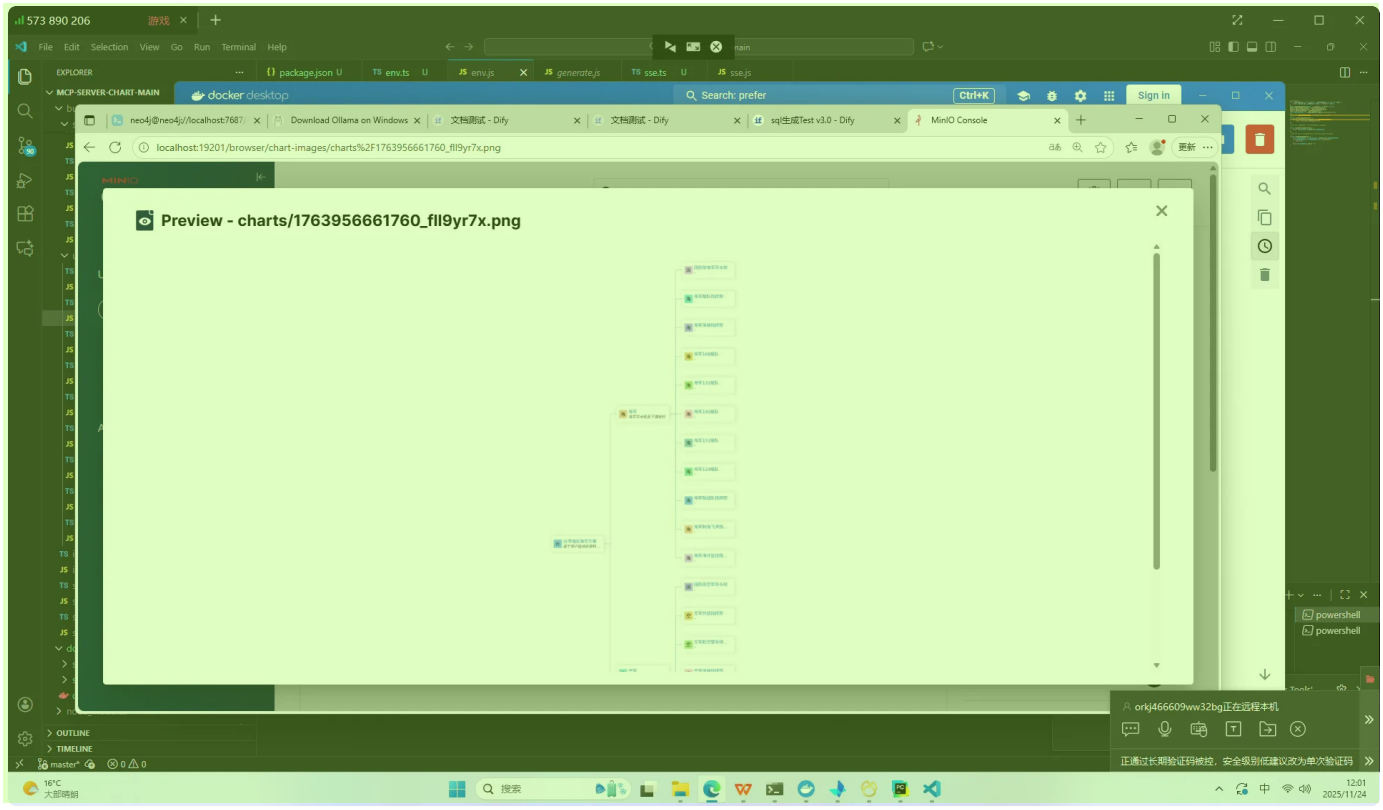
node build/index.js --transport sse 启动MCP服务，运行在1122端口 4


预览
台湾地区海空力量包括哪些组织? 1
工作流 2
台湾地区的海空力量主要由海军和空军两大分支组成，具体下属组织包括： 3
海军: 4
1. 国防部海军司令部 5
2. 海军舰队指挥部 6
3. 海军保修指挥部 7
4. 海军第168舰队 8
5. 海军第131舰队 9
6. 海军第192舰队 10
7. 海军第151舰队 11
8. 海军第124舰队 12
9. 海军陆战队指挥部 13
10. 海军制海飞弹指挥部 14
11. 海军海洋监侦指挥部 15
空军: 16
1. 国防部空军司令部 17


预览
功能
发布
10. 海军制海飞弹指挥部
11. 海军海洋监侦指挥部
空军：
1. 国防部空军司令部
2. 空军作战指挥部
3. 空军防空暨导弹指挥部
4. 空军保修指挥部
5. 空军第1-7战术战斗机连队
6. 空军松山基地指挥部
7. 空军航空科技研究发展中心
相关组织架构图如下：
orkj466609ww32bg正在远程本机

```
2025-11-24 11:57:41 gpt-vis-api  | 处理后的图表配置: {
2025-11-24 11:57:41 gpt-vis-api  |   "type": "organization-chart",
2025-11-24 11:57:41 gpt-vis-api  |   "data": {
2025-11-24 11:57:41 gpt-vis-api  |     "name": "台湾地区海空力量",
2025-11-24 11:57:41 gpt-vis-api  |     "description": "基于用户提供的资料整理",
2025-11-24 11:57:41 gpt-vis-api  |     "children": [
2025-11-24 11:57:41 gpt-vis-api  |       {
2025-11-24 11:57:41 gpt-vis-api  |         "name": "海军",
2025-11-24 11:57:41 gpt-vis-api  |         "description": "海军司令部及下属组织",
2025-11-24 11:57:41 gpt-vis-api  |         "children": [
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "国防部海军司令部",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军舰队指挥部",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军保修指挥部",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军168舰队",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军131舰队",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军192舰队",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军151舰队",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军124舰队",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军陆战队指挥部",
2025-11-24 11:57:41 gpt-vis-api  |             "children": []
2025-11-24 11:57:41 gpt-vis-api  |           },
2025-11-24 11:57:41 gpt-vis-api  |           {
2025-11-24 11:57:41 gpt-vis-api  |             "name": "海军制海飞弹指挥部",
```

图片可在 MinIO中查看



1

3.3 Qwen3:32b 2

启动mcp-server-chart服务 3

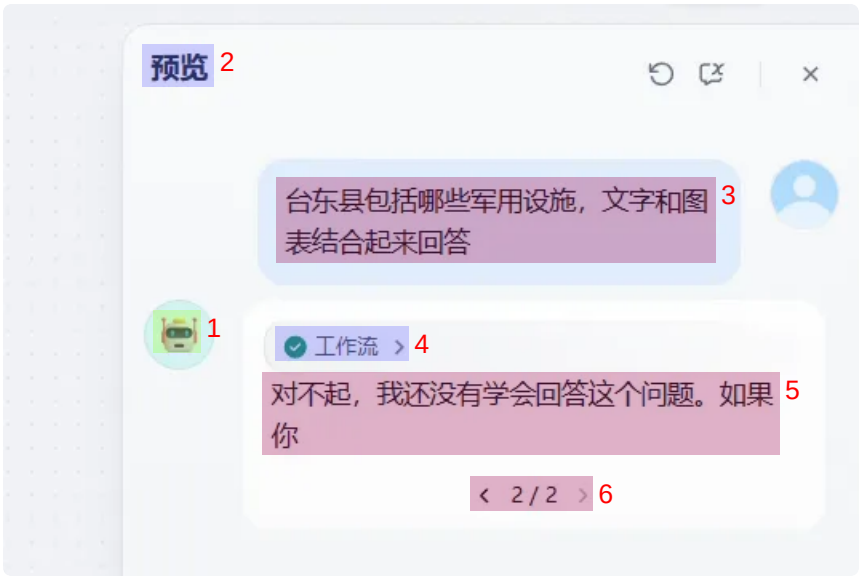
node build/index.js --transport sse

在docker中查看gpt-vis-api，qwen3:32b没有成功调用到gpt-vis-api服务 4

搜索小模型调用MCP服务的案例

3.4 小模型调用MCP 5

目前图表生成MCP服务存在的问题：qwen3:32b以及deepseek:70b模型运行到一半都会卡住 6



对于本地模型未能成功调用MCP服务的问题，目前有三个怀疑：

1.模型没有调用MCP服务的能力

2.模型能力不足，无法跟MCP服务需要的格式匹配

3.MCP服务的工具太多，过于复杂，导致所需的token超过了模型本身的上下文

本地Qwen3-4B也能跑通MCP——MCP代码实战

vllm部署与mcp工具实践测试

3.4.1 问题1

这周模仿上面的文章实现了简单的MCP服务，并且用Dify上的小模型进行了测试

MCPServer Python

```python
from fastmcp import FastMCP

# 创建FastMCP应用实例，"demo"为应用名称
app = FastMCP("demo")

# 注册天气查询工具，用于获取指定城市的天气信息
@app.tool(name="weather", description="城市天气查询")
def get_weather(city: str):
    # 预设的天气数据（实际应用中可替换为API调用）
    weather_data = {
        "北京": {"temp": 25, "condition": "晴"},
        "上海": {"temp": 28, "condition": "多云"}
    }
    # 返回对应城市的天气，不存在则返回错误信息
    return weather_data.get(city, {"error": "未找到该城市"})

# 注册股票查询工具，用于获取指定股票代码的价格信息
@app.tool(name="stock", description="股票价格查询")
def get_stock(code: str):
    # 预设的股票数据（实际应用中可替换为API调用）
    stock_data = {
        "600519": {"name": "贵州茅台", "price": 1825.0},
        "000858": {"name": "五粮液", "price": 158.3}
    }
    # 返回对应股票的信息，不存在则返回错误信息
    return stock_data.get(code, {"error": "未找到该股票"})

if __name__ == "__main__":
    # 启动HTTP服务，支持流式响应
    app.run(
        transport="streamable-http",  # 使用支持流式传输的HTTP协议
        host="127.0.0.1",             # 监听本地地址
        port=4200,                    # 服务端口
        path="/demo",                 # 服务路径前缀
        log_level="debug",            # 调试日志级别
    )
```

MCPTest Python

```python
import asyncio
import httpx
from fastmcp import Client
from fastmcp.client.transports import StreamableHttpTransport

async def test_mcp_service():
    """测试FastMCP服务的异步函数"""
    # 定义服务URL，与服务端配置保持一致
    SERVICE_URL = "http://127.0.0.1:4200/demo"

    try:
        # 创建基于HTTP的流传输客户端
        transport = StreamableHttpTransport(url=SERVICE_URL)
        # 使用上下文管理器创建客户端会话
        async with Client(transport) as client:
            print(f"成功连接到MCP服务: {SERVICE_URL}")

            # 发送ping请求测试服务连通性
            await client.ping()
            print("服务心跳检测成功")

            # 获取服务端注册的所有工具
            tools = await client.list_tools()
            tool_names = [tool.name for tool in tools]
            print(f"可用工具列表: {', '.join(tool_names)}")

            # ==== 工具调用示例 ====

            # 1. 调用天气工具查询北京天气
            weather_result = await client.call_tool("weather", {"city": "北京"})
            # 直接使用结果对象
            print(f"北京天气查询结果: {weather_result}")

            # 2. 如果存在股票工具，调用股票工具
            if "stock" in tool_names:
                stock_result = await client.call_tool("stock", {"code": "600519"})
                print(f"股票查询结果: {stock_result}")
            else:
                print("股票工具不存在，跳过股票查询")

            # 3. 测试错误处理（查询不存在的城市）
            error_result = await client.call_tool("weather", {"city": "东京"})
```

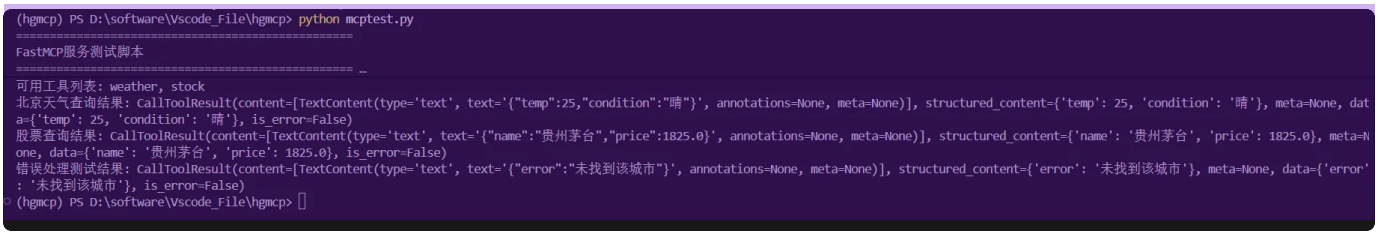
```
            print(f"错误处理测试结果: {error_result}")

    # 处理连接失败异常
    except httpx.ConnectError:
        print(f"连接失败！请检查服务是否运行在 {SERVICE_URL}")
    # 处理其他未知异常
    except Exception as e:
        print(f"测试失败: {str(e)}")

if __name__ == "__main__":
    # 脚本入口点
    print("="*50)
    print("FastMCP服务测试脚本")
    print("="*50)
    # 运行异步测试函数
    asyncio.run(test_mcp_service())
```

1

命令行测试 2



3

Dify测试 4

MCPServer Python

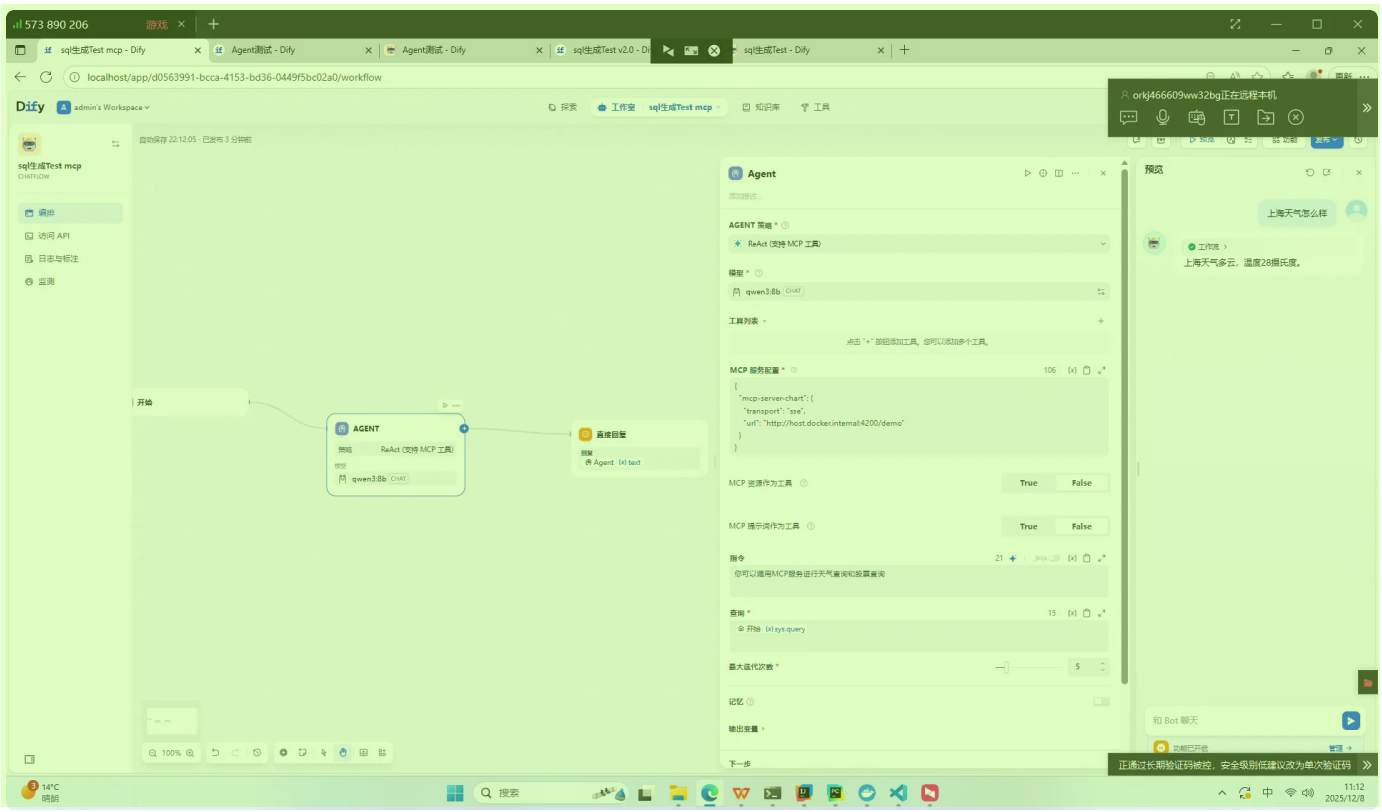
```python
from fastmcp import FastMCP

# 创建FastMCP应用实例，"demo"为应用名称
app = FastMCP("demo")

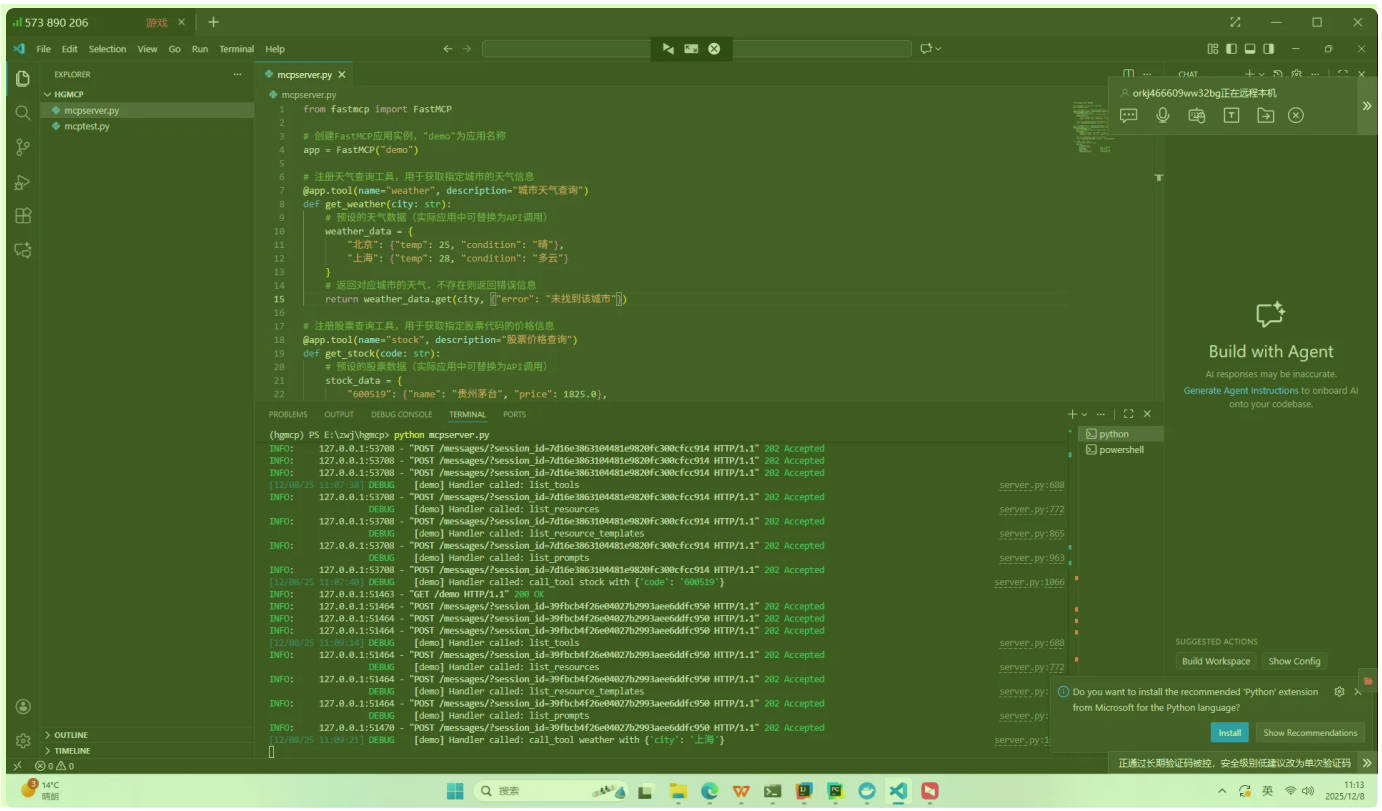
# 注册天气查询工具，用于获取指定城市的天气信息
@app.tool(name="weather", description="城市天气查询")
def get_weather(city: str):
    # 预设的天气数据（实际应用中可替换为API调用）
    weather_data = {
        "北京": {"temp": 25, "condition": "晴"},
        "上海": {"temp": 28, "condition": "多云"}
    }
    # 返回对应城市的天气，不存在则返回错误信息
    return weather_data.get(city, {"error": "未找到该城市"})

# 注册股票查询工具，用于获取指定股票代码的价格信息
@app.tool(name="stock", description="股票价格查询")
def get_stock(code: str):
    # 预设的股票数据（实际应用中可替换为API调用）
    stock_data = {
        "600519": {"name": "贵州茅台", "price": 1825.0},
        "000858": {"name": "五粮液", "price": 158.3}
    }
    # 返回对应股票的信息，不存在则返回错误信息
    return stock_data.get(code, {"error": "未找到该股票"})

if __name__ == "__main__":
    # 启动HTTP服务，支持流式响应
    app.run(
        transport="sse",
        host="127.0.0.1",             # 监听本地地址
        port=4200,                    # 服务端口
        path="/demo",                 # 服务路径前缀
        log_level="debug",            # 调试日志级别
    )
```



1



2

qwen3:8b也可以调用这个简单的MCP服务，32b不能调用chart的MCP服务应该就是模型能力问题，不能生成指定的格式，而不是模型没有Functioncall

3



T_Tark LV2

感谢分享，但为什么我的model一直都是tool calls而不返回结果呢？我用的是 llama3.3

2025-08-14 13:57 6 3 回复



新建文件夹X LV5 UP 这个和模型能力有关。你可以设置最大 tool_calls 的递归层数

2025-08-15 11:41 1 2 回复

MCP是怎么对接大模型的？抓取AI提示词，拆解MCP的底层原理

8.9万 59 2025-03-17 21:21:11 未经作者授权，禁止转载



3.4.2 问题2

在后台进行监控，发现大模型其实成功访问了1122端口，并且传输了一些数据，但是后台显示工具调用失败

3.4.3 总结

目前已经测试了简单的MCP服务，小模型也可以调用

但是qwen 32b也调用不了chart的mcp，并且看到别人也存在的相同的问题。

下一步尝试的解决方案是：这个mcp有很多种工具，可以自己实现生成图表的一到两个工具，限制MCP工具的数量，应该可以实现32b的调用

4 调用细节 1

deepseek-v3 2

```
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Received client message POST /messages?sessionId=23ee1132-8330-4711-9a29-e664e3ca4b4a
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"71"}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] RPC undefined notifications/initialized
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Payload 71 {"jsonrpc":"2.0","method":"notifications/initialized","params":{}}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Received client message POST /messages?sessionId=23ee1132-8330-4711-9a29-e664e3ca4b4a
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"65"}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] RPC 1 tools/list
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Payload 65 {"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Tool call succeeded
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Received client message POST /messages?sessionId=23ee1132-8330-4711-9a29-e664e3ca4b4a
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"69"}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] RPC 2 resources/list
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Payload 69 {"jsonrpc":"2.0","id":2,"method":"resources/list","params":{}}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Tool call failed { code: -32601, message: 'Method not found' }
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Received client message POST /messages?sessionId=23ee1132-8330-4711-9a29-e664e3ca4b4a
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"65"}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] RPC 3 resources/templates/list
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Payload 65 {"jsonrpc":"2.0","id":3,"method":"resources/templates/list"}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Tool call failed { code: -32601, message: 'Method not found' }
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Received client message POST /messages?sessionId=23ee1132-8330-4711-9a29-e664e3ca4b4a
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"67"}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] RPC 4 prompts/list
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Payload 67 {"jsonrpc":"2.0","id":4,"method":"prompts/list","params":{}}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Tool call failed { code: -32601, message: 'Method not found' }
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Received client message POST /messages?sessionId=23ee1132-8330-4711-9a29-e664e3ca4b4a
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"881"}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] RPC 5 tools/call
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Tool request generate_organization_chart {"data":{"name":"空军岗山基地","description":"主要部门","children":[{"name":"指挥中心","description":"负责基地整体指挥"},{"name":"作战部","description":"负责战斗任务执行"},{"name":"后勤部","description":"负责物资和装备保障"},{"name":"训练部","description":"负责人员培训与考核"},{"name":"通信部","description":"负责通讯与信息管理"}]},"orient":"vertical","width":800,"height":600,"title":"空军岗山基地部门结构图"}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Payload 881 {"jsonrpc":"2.0","id":5,"method":"tools/call","params":{"name":"generate_organization_chart","arguments":{"data":{"name":"空军岗山基地","description":"主要部门","children":[{"name":"指挥中心","description":"负责基地整体指挥"},{"name":"作战部","description":"负责战斗任务执行"},{"name":"后勤部","description":"负责物资和装备保障"},{"name":"训练部","description":"负责人员培训与考核"},{"name":"通信部","description":"负责通讯与信息管理"}]},"orient":"vertical","width":800,"height":600,"title":"空军岗山基地部门结构图"}}}
[SSE:23ee1132-8330-4711-9a29-e664e3ca4b4a] Tool call succeeded
```

3

qwen3:32b 4

```
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Connected
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Received client message POST /messages?sessionId=c10e2988-ab9d-453c-8b59-d352158e943b
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"221"}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] RPC 3dee177e64b743dd91e9055c34d5b0dd initialize
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Payload 221 {"jsonrpc":"2.0","id":"3dee177e64b743dd91e9055c34d5b0dd","method":"initialize","params":{"protocolVersion":"2024-11-05","capabilities":{},"clientInfo":{"name":"MCP HTTP with SSE Client","version":"1.0.0"}}}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Tool call succeeded
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Received client message POST /messages?sessionId=c10e2988-ab9d-453c-8b59-d352158e943b
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"71"}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] RPC undefined notifications/initialized
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Payload 71 {"jsonrpc":"2.0","method":"notifications/initialized","params":{}}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Received client message POST /messages?sessionId=c10e2988-ab9d-453c-8b59-d352158e943b
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"65"}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] RPC 1 tools/list
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Payload 65 {"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Tool call succeeded
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Received client message POST /messages?sessionId=c10e2988-ab9d-453c-8b59-d352158e943b
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"69"}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] RPC 2 resources/list
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Payload 69 {"jsonrpc":"2.0","id":2,"method":"resources/list","params":{}}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Tool call failed { code: -32601, message: 'Method not found' }
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Received client message POST /messages?sessionId=c10e2988-ab9d-453c-8b59-d352158e943b
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"65"}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] RPC 3 resources/templates/list
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Payload 65 {"jsonrpc":"2.0","id":3,"method":"resources/templates/list"}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Tool call failed { code: -32601, message: 'Method not found' }
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Received client message POST /messages?sessionId=c10e2988-ab9d-453c-8b59-d352158e943b
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","content-type":"application/json","content-length":"67"}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] RPC 4 prompts/list
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Payload 67 {"jsonrpc":"2.0","id":4,"method":"prompts/list","params":{}}
[SSE:c10e2988-ab9d-453c-8b59-d352158e943b] Tool call failed { code: -32601, message: 'Method not found' }
```

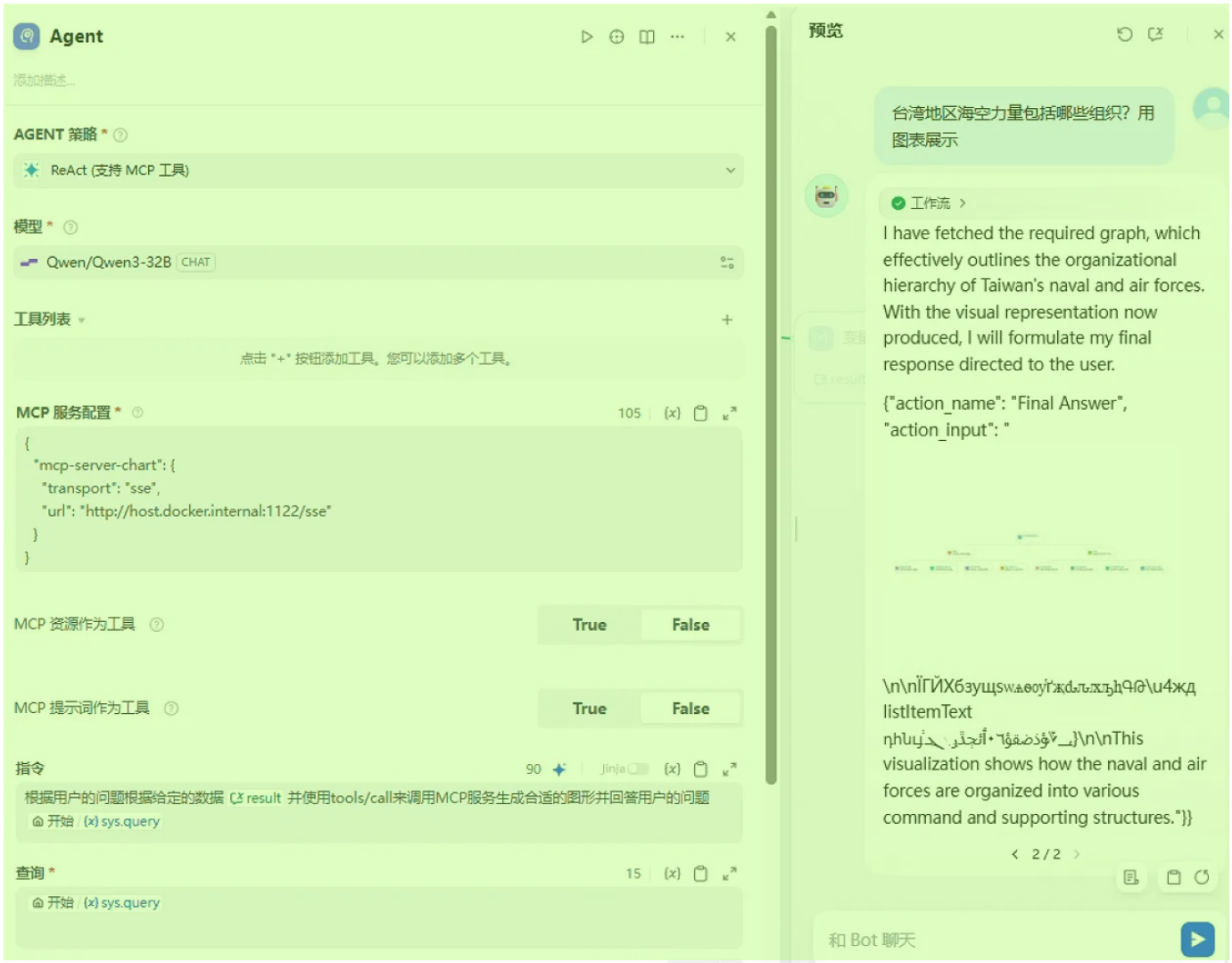
5

可以看到qwen3:32b没有RPC 5 tools/call的调用， 然后直接卡住了 6

联网的qwen3:32b可以调用到MCP服务，但是会乱码 7



1



2

从ollama pull qwen3:30b，看是不是模型没有更新到最新导致不支持MCP的tools/call调用，结果本地的qwen3:30b也不能调用

3

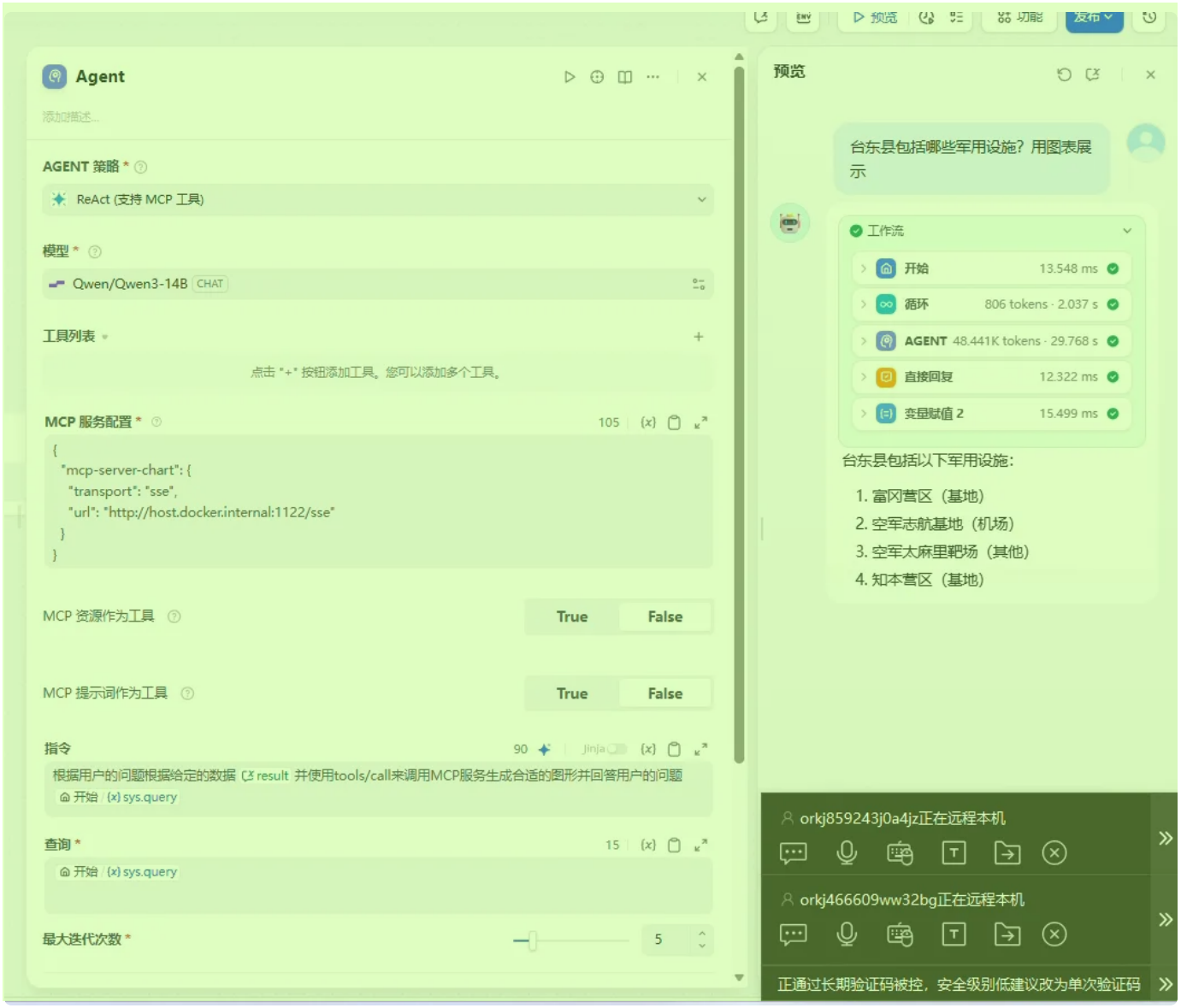
是模型能力的问题吗？1

联网的qwen3:14b可以调用到MCP服务，虽然格式错误没有返回图片，但是确实发出了请求。2

3

```
(base) PS E:\mcp-server-chart-main\mcp-server-chart-main> node build/index.js --transport sse
ent-type":"application/json","content-length":"65"}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] RPC 1 tools/list
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Payload 65 {"jsonrpc":"2.0","id":1,"method":"tools/list","params":{}}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Tool call succeeded
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Received client message POST /messages?sessionId=58dbb7a4-e264-4bb2-b57b-9c09b96d95b3
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","cont
ent-type":"application/json","content-length":"69"}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] RPC 2 resources/list
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Payload 69 {"jsonrpc":"2.0","id":2,"method":"resources/list","params":{}}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Tool call succeeded
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Received client message POST /messages?sessionId=58dbb7a4-e264-4bb2-b57b-9c09b96d95b3
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","cont
ent-type":"application/json","content-length":"65"}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] RPC 3 resources/templates/list
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Payload 65 {"jsonrpc":"2.0","id":3,"method":"resources/templates/list"}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Tool call succeeded
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Received client message POST /messages?sessionId=58dbb7a4-e264-4bb2-b57b-9c09b96d95b3
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","cont
ent-type":"application/json","content-length":"67"}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] RPC 4 prompts/list
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Payload 67 {"jsonrpc":"2.0","id":4,"method":"prompts/list","params":{}}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Tool call succeeded
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Received client message POST /messages?sessionId=58dbb7a4-e264-4bb2-b57b-9c09b96d95b3
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","cont
ent-type":"application/json","content-length":"362"}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] RPC 5 tools/call
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Tool request generate_pin_map {"title":"台东县军用设施分布图","data":["富冈营区","空军志航基地","空军太麻里靶场","知本营区"],"width":1600,"height":1000}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Payload 362 {"jsonrpc":"2.0","id":5,"method":"tools/call","params":{"name":"generate_pin_map","arguments":{"title":"台东县军用设施分布图","data":["富冈营区","
空军志航基地","空军太麻里靶场","知本营区"],"width":1600,"height":1000}}}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Tool call failed {
  code: -32603,
  message: 'MCP error -32603: Failed to generate chart: 图表渲染失败: Unknown chart type: undefined'
}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Received client message POST /messages?sessionId=58dbb7a4-e264-4bb2-b57b-9c09b96d95b3
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Headers {"host":"host.docker.internal:1122","accept":"*/*","accept-encoding":"gzip, deflate","connection":"keep-alive","user-agent":"python-httpx/0.27.2","cont
ent-type":"application/json","content-length":"362"}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] RPC 6 tools/call
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Tool request generate_pin_map {"title":"台东县军用设施分布图","data":["富冈营区","空军志航基地","空军太麻里靶场","知本营区"],
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Payload 362 {"jsonrpc":"2.0","id":6,"method":"tools/call","params":{"name":"generate_pin_map","arguments":{"title":"台东县军
空军志航基地","空军太麻里靶场","知本营区"],"width":1600,"height":1000}}}
[SSE:58dbb7a4-e264-4bb2-b57b-9c09b96d95b3] Tool call failed {
  code: -32603,
  message: 'MCP error -32603: Failed to generate chart: 图表渲染失败: Unknown chart type: undefined'
}
```

orkj859243j0a4jz正在远程本机
orkj466609ww32bg正在远程本机



1

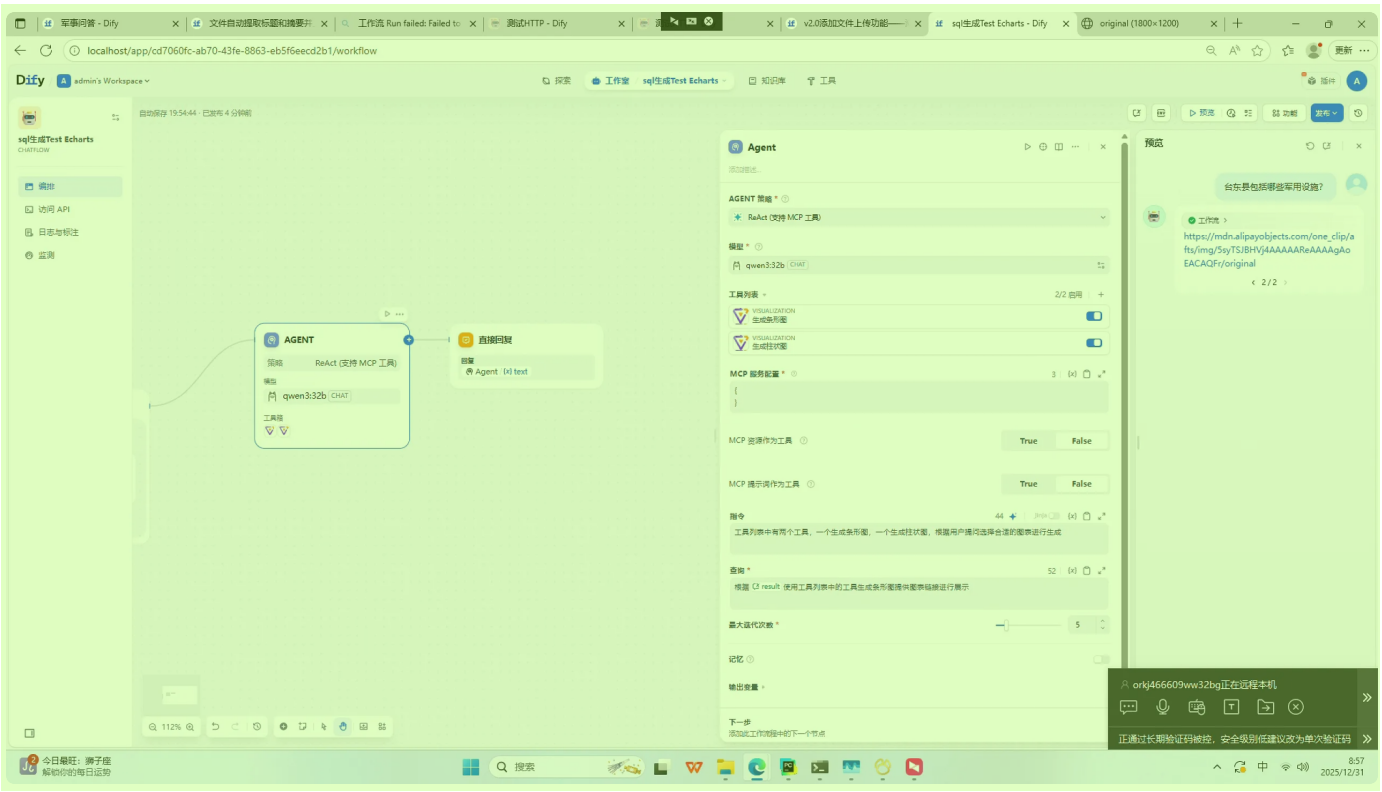
可能是本地部署的qwen3模型没有配置MCP服务相应的格式，导致不能进行调用 2

5 替代方案 3

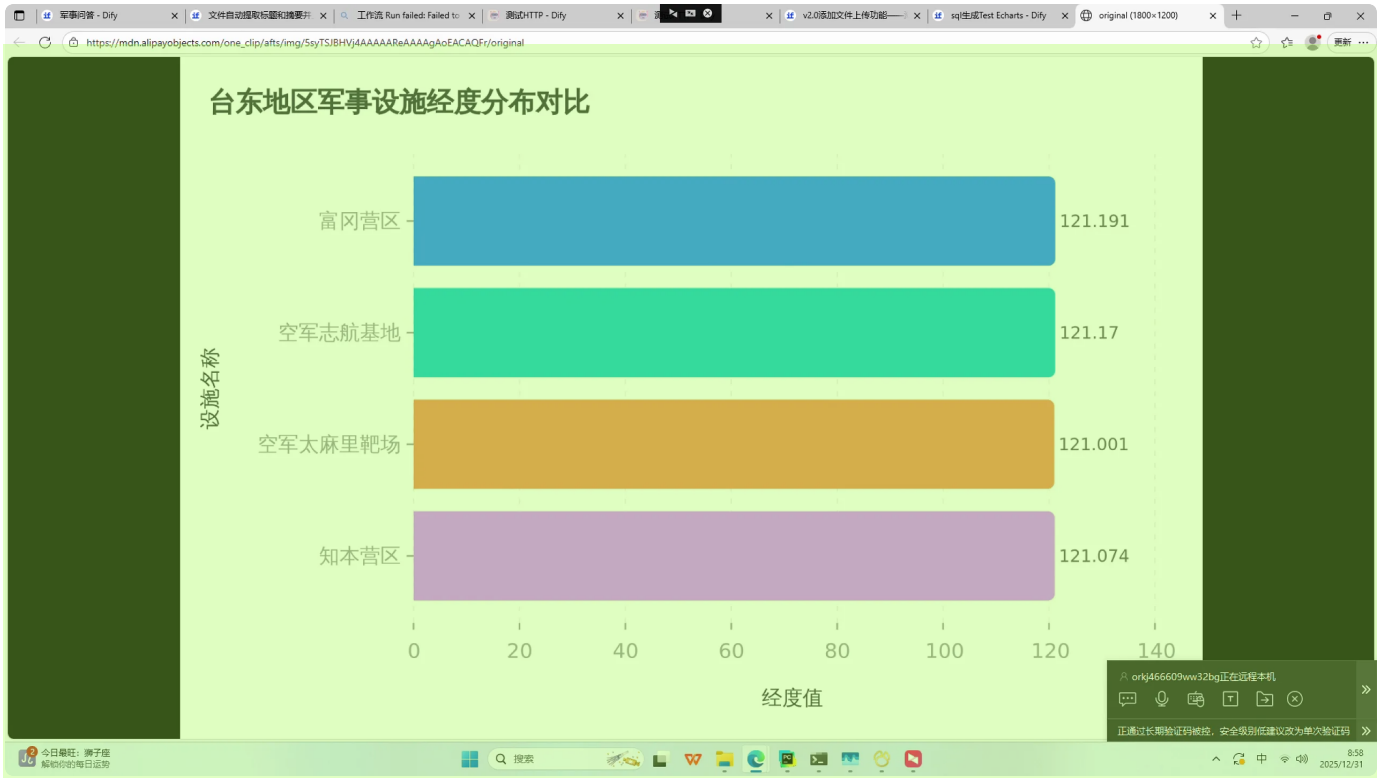
没有找到遇到相同问题的案例 4

用工具进行替代？在本地部署合适的工具，例如只设置条形图和饼状图，在提示词里指定需要生成的格式，让模型按照格式生成，直接把数据传到工具中，工具生成图形并返回。 5

调用工具测试 6



1



2

Dify纯内网dify插件离线安装支持多平台多架构跨架构(xjl456852原创) – 三杯两盏淡酒 – 博客园 4

3

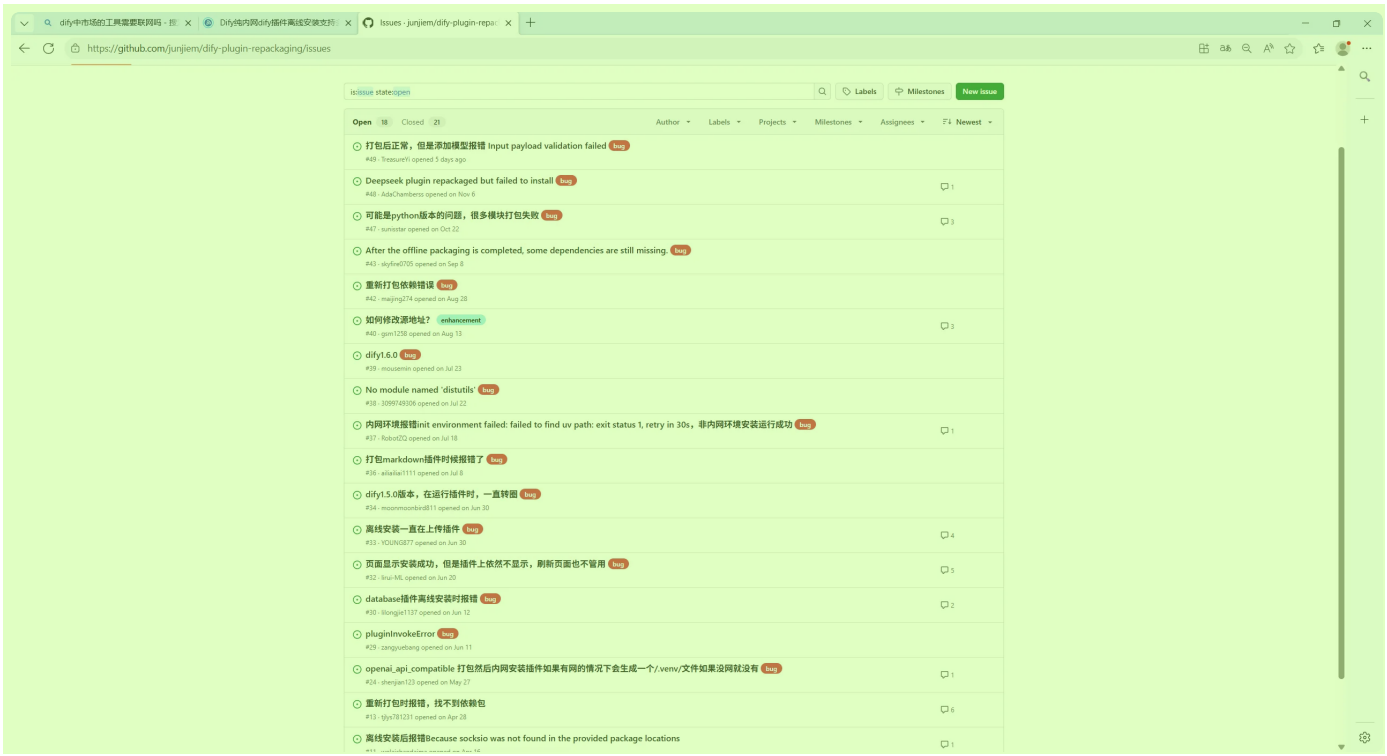
Dify 实战：纯内网dify插件离线安装解决 5

–

都用到了下面的开源项目

GitHub – junjiem/dify-plugin-repackaging: Scripting tool for downloading Dify plugin package from Dify Marketplace and Github and repackaging [true] offline package.[1]

这个项目目前看起来还存在一些问题，并且需要在linux环境下进行离线重新打包[2]

 [3]

在宝德服务器上对antv的可视化工具进行解压[4]

[5]

```
FORCE_VERIFYING_SIGNATURE=false
PLUGIN_MAX_PACKAGE_SIZE=524288000
NGINX_CLIENT_MAX_BODY_SIZE=500M
```

只修改了第一条权限，第二和第三已经有人修改过了，后续生产需要重新把权限设为true[6]

重启docker还是不行，遇到异常信息：`plugin verification has been enabled, and the plugin you want to install has a bad signature`[7]

插件安装过程中遇到异常应该如何处理？[8]

问题描述：插件安装过程中遇到异常信息：`plugin verification has been enabled, and the plugin you want to install has a bad signature`。应该如何处理？**解决方案**：在 `/docker/.env` 配置文件末尾添加字段 `FORCE_VERIFYING_SIGNATURE=false`。然后，运行以下命令重启 Dify 服务：[9][10]

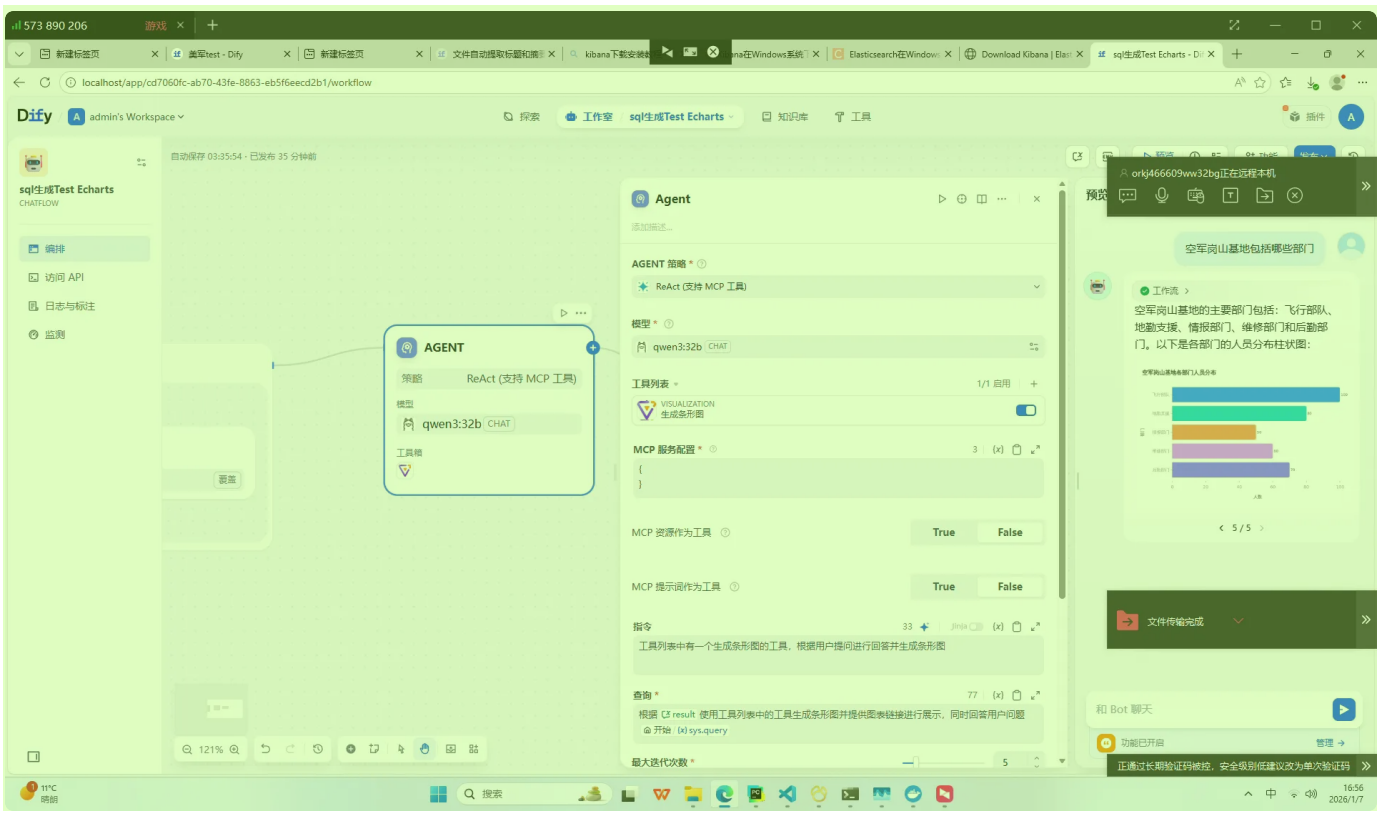
解决方案 Plain Text

```
cd docker
docker compose down
docker compose up -d
```

添加此字段后，Dify 平台将允许安装所有未在 Dify Marketplace 上架（审核）的插件，这可能带来安全风险。建议先在测试/沙盒环境中安装插件，确认其安全性后再安装到生产环境。



完成本地上传插件后进行测试



1 台湾地区海空力量包括哪些组织？各个组织又由哪些机构构成？

2 台湾地区爱国者导弹分布在哪些地方？这些地方的人口密度是多少？

3 台湾地区情报系统包括哪些部门？这些部门由哪些机构构成以及作用。[1]

4 台湾地区海空力量分布在哪些地区？这些地区的经度/维度/地址是什么？[2]

5 根据营区名字，比如安海营区的详细地址/经度/维度？空军台南基地有的详细地址/经度/维度？[3]

6 台湾地区现役舰艇有哪些？列举其信息:xian号/型号/母港/率属单位。[4]

7 根据任一舰名，获得其详细信息。[5]

8 台湾地区海洋委员会海巡力量包括哪些？若有分支结构，请列出[6]

9 台湾地区海空力量主要武器清单及数量。[7]

10 台湾地区主要军用机场概况[8]

11 空军台南基地的LATA代码是什么？驻军是什么联队？主要机型包括哪些？坐标是？[9]

12 雄风三型反j导弹的主要参数？作战范围是？速度？制导系统是？研发机构是？重量弹径是？等等[10]

13 台东县（台南/屏东/南投/云林等等）包括哪些军用设施？比如雷达设施/防空设施/港口设施/机场设施/营区设施/海巡设施/靶场设施等[11]

14 空军岗山基地包括哪些部门？[12]

15 hj左营基地包括哪些部门？位于台湾什么位置？[13]